WaterPoint Watch

A Community Water-Point Fault Reporting and Repair-Prioritization System

Course: PROG101 — Principles of Programming Logic and Design
Assignment: Designing Logical Solutions for Sierra Leone Community Challenges
Student name and ID: Complete before submission
Institution: Limkokwing University of Creative Technology, Sierra Leone
Semester: September 2026–January 2027
Submission date: Complete before submission

> Before submitting, replace the two student-information instructions above with your own details. The sample scenarios in this report are fictional and are for demonstrating the logic only.

1. Problem description

Reliable access to safe water remains a community concern in Sierra Leone. UNICEF Sierra Leone describes gaps in reliable WASH services, especially in underserved communities, and reports that about 65 per cent of people have access to basic water services on its country WASH page [1]. Globally, 2.1 billion people still lacked safely managed drinking water in 2024 [3]. This assignment focuses on one manageable part of that wider challenge: helping community representatives record water-point faults and helping district WASH staff decide which reported repairs should be reviewed first.

A fault may be a broken hand pump, a serious leak, reduced or intermittent flow, or suspected contamination. If reports are scattered across conversations and notebooks, staff may have difficulty comparing urgency, identifying the number of people affected, and seeing which issue has remained unresolved longest. This design does not claim that every community currently lacks a reporting process. It proposes a simple shared queue as a practical way to organize reports and make triage criteria visible.

1.1 Who is affected

The primary beneficiaries are households that depend on shared community water points, including children, older people, people with disabilities, and other residents who have fewer alternatives. Community water committees and local WASH staff benefit from a consistent

record of faults. District coordinators and repair teams benefit from a ranked work queue rather than an unstructured list.

1.2 Why the problem exists

Water infrastructure can fail through wear, damage, leaks, or environmental risks. The response also depends on timely reporting, verification, available technicians, spare parts, transport, and coordination. UNICEF notes that financing gaps, shortages of skilled personnel, and climate risks can slow WASH progress in Sierra Leone [1]. The proposed software cannot fix these structural constraints. It can make reported needs easier to record, compare, assign, and follow up.

1.3 Proposed solution

WaterPoint Watch is a design for a low-bandwidth, offline-capable reporting and triage tool. A community reporter records the water-point identifier, issue, severity, estimated number of people affected, whether vulnerable users depend on the point, and how many days the issue has remained open. The tool validates the report, calculates a transparent priority score, flags suspected contamination for precautionary review, and displays an ordered repair queue. When connectivity returns, saved records can be synchronized with a district view.

The tool supports human decision-making. It does not diagnose water quality, dispatch a repair team automatically, or replace a water-quality test or decision by qualified public-health staff. Suspected contamination is treated as a precautionary signal for prompt verification and referral, not proof that water is contaminated.

2. Objectives and scope

The system aims to make fault reporting consistent, prioritize cases using documented rules, support follow-up from report to closure, work when connectivity is intermittent, and avoid collecting unnecessary personal information. The design aligns most directly with SDG 6, including Target 6.1 on equitable access to safe and affordable drinking water and Target 6.b on community participation in water and sanitation management [2].

2.1 In scope

- Create, validate, and save a water-point fault report.
- Assign each report a unique report ID and a status.
- Calculate an urgency score and category using the rules in Section 5.
- Place open reports in a deterministic priority order.

- Update a report through verification, assignment, repair, and closure.
- Work offline for data entry and queue viewing, then synchronize later.
- Provide a simple audit trail of status changes and timestamps.

2.2 Out of scope

- Testing or certifying water safety.
- Dispatching technicians, purchasing parts, or guaranteeing repair times.
- Replacing government water-point registers or existing WASH coordination processes.
- Publicly displaying private reporter details or precise household locations.
- Sending real SMS or emergency alerts in this logic-only assignment.

3. Functional and quality requirements

3.1 Functional requirements

1. The user can select a known water-point ID or request that an authorized coordinator add a missing point.
2. The user can select an issue type and a severity level from 1 to 5.
3. The user must enter a positive estimate of people affected, the number of days the issue has been open, and whether vulnerable users depend on the water point.
4. The system checks required fields and rejects invalid values with a clear correction message.
5. The system assigns an ID, date, time, initial status, score, and priority category to each accepted report.
6. A severity-5 report, defined here as suspected contamination, receives a Critical category and a precautionary verification flag regardless of its numeric score.
7. Authorized users can update report status and add a short repair note without entering personal health information.
8. The system sorts open reports by priority category, then score, then age, then report ID.
9. The system retains offline entries and synchronizes them when a connection is available. Conflicts must be shown to an authorized reviewer instead of silently overwriting an update.
10. The system can show a summary of open, assigned, in-progress, and resolved reports.

3.2 Quality and safety requirements

The interface should use plain language, readable text, and a small number of fields so that it can be used on basic devices. Data entry should work offline. The system should record only what is needed for triage, restrict edit access by role, and keep a basic change history. A suspected contamination flag must clearly state that the report requires verification and must not present an untested source as certified safe or unsafe. Before real deployment, local WASH and public-health authorities should review the severity scale, precautionary wording, access controls, and escalation procedure.

4. Logical design and data

4.1 Inputs, processing, and outputs

Inputs: water-point ID, community/ward, issue type, severity (1–5), estimated people affected, vulnerable-users flag, days open, reporter role, date/time, and optional non-sensitive note.

Processing: validate fields; check for a matching open report; calculate a score; apply the contamination override; assign category and recommended review; save status history; and sort the queue.

Outputs: confirmation with report ID, score and priority category; precautionary verification flag where applicable; ranked open-work queue; status summaries; and a report history.

4.2 Variables and data types

- `report_id` : text, unique identifier.
- `water_point_id` : text, identifier from the local water-point register.
- `issue_type` : enumerated text such as `pump_failure` , `leak` , `low_flow` , or `suspected_contamination` .
- `severity` : integer from 1 to 5.
- `people_affected` : positive integer estimate.
- `vulnerable_users` : Boolean (`true` or `false`).
- `days_open` : non-negative integer.
- `score` : integer from 5 to 39 under the proposed formula.
- `priority` : enumerated text: `Critical` , `Urgent` , `High` , `Medium` , or `Low` .
- `status` : enumerated text: `New` , `Verified` , `Assigned` , `In Progress` , `Resolved` , or `Closed` .

- `created_at`, `updated_at`: date/time values.
- `notes`: optional short text, with a warning not to include sensitive personal details.

4.3 Decision and repetition structures

The system uses selection (`IF/ELSE`) to validate values, detect suspected contamination, calculate a priority category, and control which status updates are allowed. It uses repetition (`FOR EACH`) to calculate or display every report in the queue. The queue is then sorted with a deterministic tie-break order so two users viewing the same records receive the same ordering.

5. Algorithm design

5.1 Severity scale

- **1 — Cosmetic:** no meaningful loss of water service; for example, a damaged sign.
- **2 — Minor:** small leak or reduced flow while the point remains usable.
- **3 — Moderate:** intermittent or unreliable service.
- **4 — Severe:** the water point is not functioning or provides no service.
- **5 — Suspected contamination:** a report of a possible water-quality hazard requiring precautionary verification.

Severity 5 does not mean contamination has been laboratory-confirmed. Only an authorized test or competent authority can determine water quality.

5.2 Priority score

For each valid report, calculate:

`score = (4 × severity) + min(8, ceiling(people_affected ÷ 25)) + min(6, days_open) + (5 if vulnerable_users else 0)`

The maximum ordinary numeric score is 35 for severity 4. The scale gives weight to service severity, affected users, waiting time, and whether vulnerable users rely on the point. The cap on affected users and days prevents either input from overwhelming the other factors. These weights are proposed design choices for the assignment, not empirically validated public-health weights; local WASH stakeholders should review them before deployment.

Assign a category in this order:

1. If `severity = 5` or `issue_type = suspected_contamination`, set `priority = Critical` and set `verification_flag = true`.

2. Otherwise, if `score ≥ 27`, set `priority = Urgent`.
3. Otherwise, if `score ≥ 18`, set `priority = High`.
4. Otherwise, if `score ≥ 9`, set `priority = Medium`.
5. Otherwise, set `priority = Low`.

Sort the open queue by category rank (`Critical`, `Urgent`, `High`, `Medium`, `Low`), then score descending, days open descending, and report ID ascending. Statuses `Resolved` and `Closed` are excluded from the open queue.

5.3 Step-by-step algorithm

1. Start and receive a fault report.
2. Check that the water-point ID exists and that required values are present.
3. Validate severity, affected-user count, days open, and issue type. If invalid, explain the error and request correction.
4. Check whether an open report already exists for the same water point and issue. If it does, attach the new observation to that report and preserve the original creation date. Otherwise, create a new report ID and record.
5. Calculate the numeric priority score using the formula above.
6. If the issue is suspected contamination, assign Critical priority and set the precautionary verification flag. Otherwise, select the category from the score thresholds.
7. Save the record and its initial status as `New`.
8. Add the report to the open queue and sort the queue using the documented tie-break order.
9. Show the report ID, category, score, and next review action to the authorized user.
10. When staff verify, assign, begin, or complete work, update status and retain the change in the history.
11. Synchronize pending offline changes when connectivity is available, showing conflicts for review.
12. Stop.

5.4 Pseudocode

```
FUNCTION calculateScore(severity, affected, daysOpen, vulnerable):
    userPoints = MIN(8, CEILING(affected / 25))
    waitPoints = MIN(6, daysOpen)
    riskPoints = 5 IF vulnerable ELSE 0
    RETURN 4*severity + userPoints + waitPoints + riskPoints

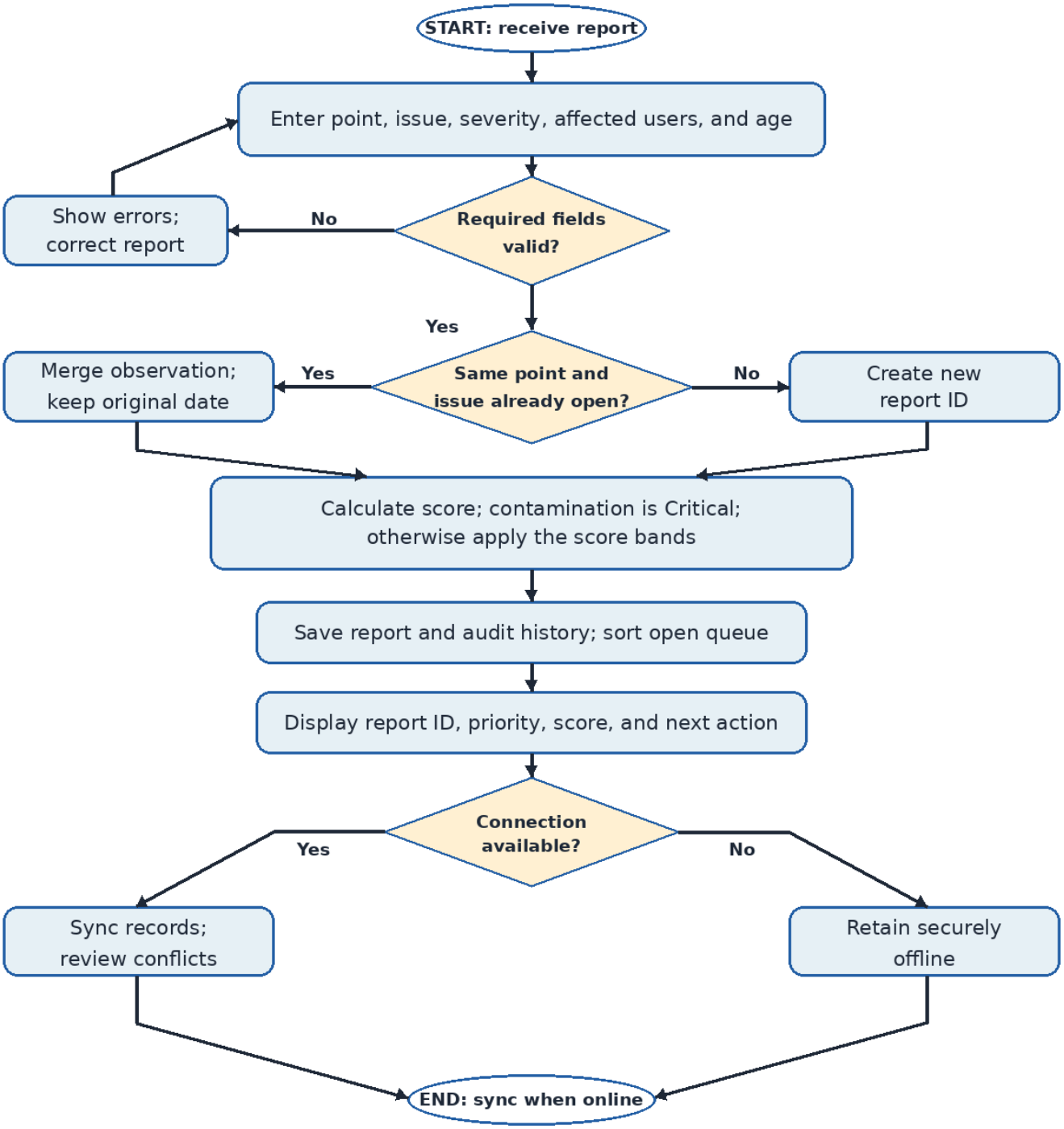
FUNCTION classify(report):
    report.score = calculateScore(
        report.severity, report.people, report.days, report.vulnerable
    )
    report.verify = false
    IF report.severity = 5 OR report.issue = suspected_contamination:
        report.priority = Critical
        report.verify = true
    ELSE IF report.score ⩾ 27: report.priority = Urgent
    ELSE IF report.score ⩾ 18: report.priority = High
    ELSE IF report.score ⩾ 9: report.priority = Medium
    ELSE: report.priority = Low
    RETURN report

FUNCTION submit(report, openQueue):
    IF required fields are missing: RETURN field error
    IF severity not 1..5 OR affected < 1 OR daysOpen < 0:
        RETURN range error
    duplicate = FIND open report with same point and issue
    IF duplicate exists:
        append observation; retain original creation date
        report = duplicate
    ELSE:
        assign unique ID; set status = New; add to openQueue
    report = classify(report)
    save report and status history
    sort by category, score descending, age descending, ID ascending
    RETURN report

FOR EACH report IN openQueue:
    DISPLAY ID, water point, priority, score, status, and age
```

5.5 Flowchart

The flowchart summarizes report intake, validation, duplicate handling, scoring, contamination escalation, and queue display.



The editable diagram source is included as `flowchart.mmd` in the project folder.

6. Functional modules

Module	Main function	Input	Output
Report Intake	Capture and normalize a report	Form fields	Structured report or validation error
Validation	Check required fields and permitted ranges	Structured report	Valid report or field-specific error
Duplicate Check	Find an open report for the same point and issue	New report and open records	Existing report to update or new-record decision
Scoring and Classification	Calculate score, category, and safety flag	Valid report	Classified report
Queue Manager	Exclude closed items and sort open reports	Report collection	Ranked open queue
Status and Audit	Enforce status transitions and record changes	Report ID, new status, user role	Updated status history
Offline Sync	Save pending changes locally and reconcile after reconnection	Local changes and server records	Synchronized records or conflict for review
Summary View	Count open, assigned, in-progress, and resolved items	Report collection	Dashboard summary

6.1 Functions

The core functions are `validateReport(report)`, `calculateScore(severity, peopleAffected, daysOpen, vulnerableUsers)`, `classifyReport(report)`, `findDuplicate(report, openReports)`, `sortQueue(reports)`, `updateStatus(reportId, newStatus, role)`, and `syncPendingRecords(localRecords, remoteRecords)`. Each function has one responsibility, which makes it easier to test and maintain the logic.

7. Expected outputs and sample scenarios

All scenarios below are fictional test data. `Affected points` use the ceiling operation in the score formula.

Report	Example input	Score calculation	Expected output
WP-101	Suspected contamination; severity 5; 120 people; 1 day open; vulnerable users yes	20 + 5 + 1 + 5 = 31	**Critical**; precautionary verification flag; notify authorized WASH/public-health contact for prompt assessment.
WP-204	Pump failure; severity 4; 90 people; 3 days open; vulnerable users yes	16 + 4 + 3 + 5 = 28	**Urgent**; add to top repair queue after any Critical reports.
WP-309	Leak; severity 2; 30 people; 0 days open; vulnerable users no	8 + 2 + 0 + 0 = 10	**Medium**; record and schedule review according to available repair capacity.

The resulting order is WP-101, WP-204, then WP-309. Critical is placed first because of the suspected-contamination override; WP-204 is Urgent because its score is 28; WP-309 is Medium because its score is 10. This is a triage order, not a promise that repairs will occur in that order or within a specific time.

7.1 Edge-case tests

Test	Input condition	Expected behavior
Missing ID	Water-point ID is blank	Reject and request a valid registered point or coordinator review.
Invalid severity	Severity is 0 or 6	Reject; accept only values 1–5.
Negative wait time	Days open is -1	Reject; days open must be 0 or greater.
Zero affected users	People affected is 0	Reject; estimate must be at least 1.
Duplicate open report	Same point and issue already has an open record	Append observation to the existing record and retain the original report date.
Suspected contamination with a small score	Severity 5 or contamination issue type	Always set Critical and the verification flag. Do not claim the water has been tested.
Closed report	Status is Resolved or Closed	Do not include in the open repair queue.
Tie in priority and score	Two open reports share category and score	Sort by days open descending, then report ID ascending.

8. Computational thinking

Problem decomposition: The larger service challenge is divided into report intake, validation, duplicate detection, scoring, classification, queue ordering, status tracking, and synchronization.

Pattern recognition: Similar fault reports share common fields and pass through the same validation and triage stages. Repeated faults at one point can be noticed through the water-point ID and issue type.

Abstraction: The assignment represents each water point and fault with only the fields needed to triage and follow up. It does not model every engineering detail of a pump or every household in the community.

Algorithm design: A clear formula and fixed thresholds turn input values into a repeatable score and category. The tie-break rules ensure a stable queue. A human coordinator retains authority over verification and repair decisions.

Testing and evaluation: Boundary-value tests check invalid values, category thresholds, the contamination override, duplicates, closed records, and ties. A pilot should compare system priorities with decisions by local WASH staff, review missed urgent cases, and adjust the scale only with documented approval.

9. Open-source and Digital Public Goods rationale

The proposed tool should be released with source code, setup instructions, data definitions, and a permissive open-source license such as MIT, selected and applied by the actual project owner before publication. The Open Source Initiative's definition includes access to source code, permission for derived works, and redistribution rights [5]. Open licensing would let Sierra Leonean schools, community organizations, local authorities, and developers inspect, adapt, translate, and maintain the tool without being locked to one vendor. Reuse could reduce duplicated development effort, although it would not remove costs for devices, training, connectivity, repairs, hosting, or local support.

The design also follows several Digital Public Goods Standard considerations: SDG relevance, open licensing, clear ownership, platform independence, documentation, data minimization, privacy, security, and standards-based practice [4]. It should not be called a recognized Digital Public Good at this stage. Recognition requires review against the Standard and registry listing; this assignment only describes how the system could be designed toward those requirements [4].

For privacy, the minimum viable system should avoid collecting names, phone numbers, or household-level health details. Community-level counts and water-point IDs are sufficient for the proposed triage logic. Role-based access, secure synchronization, backups, retention rules, and a documented incident process would still be needed before deployment.

10. Sierra Leone community impact and sustainability

The expected benefit is better visibility of water-point failures and a more explainable way to organize repair requests. Community users can see that their report was received and can refer to its ID when asking for an update. WASH coordinators can identify older, more severe, and broader-impact reports. The design supports local community participation in management, consistent with SDG Target 6.b, and contributes to the wider goal of equitable access to safe drinking water under Target 6.1 [2].

The tool is intentionally small and offline-capable because network access may be intermittent in some settings. Its long-term value would depend on local ownership: an agreed water-point register, designated data stewards, trained users, a functioning repair process, clear safety escalation, and maintenance funding. A pilot should be conducted with one willing community and its responsible WASH authority, using fictional or approved test cases first. Accessibility, local language needs, data retention, and the numeric scoring weights should be reviewed with intended users before the system handles real reports.

11. Limitations and risks

A numerical score simplifies a complex decision and can hide local context. Estimated user counts may be uncertain, and the score does not include distance, spare-part availability, seasonal access, or engineering difficulty. A high score does not mean that a technician is available. The contamination flag is not a water test. Incorrect or malicious reports are possible, so verification and audit history matter. Offline records can conflict during synchronization, so changes must be reviewed rather than silently discarded. These risks mean the tool should inform—not replace—local human judgment and official procedures.

12. Conclusion

WaterPoint Watch demonstrates how programming logic can support a real community need through structured inputs, validation, arithmetic, decisions, repetition, modular functions, and explicit outputs. Its main design choice is to separate ordinary repair prioritization from the precautionary handling of suspected contamination. The report, flowchart, and source-ready files provide a complete logic design for review and possible local adaptation. Before real use, local authorities should validate the workflow, scoring weights, privacy controls, and safety guidance.

References

1. UNICEF Sierra Leone. “Water, Sanitation and Hygiene (WASH).” Source page.
2. United Nations Department of Economic and Social Affairs. “Goal 6: Ensure availability and sustainable management of water and sanitation for all.” Source page.
3. WHO/UNICEF Joint Monitoring Programme. “Progress on household drinking water, sanitation and hygiene 2000–2024: Special focus on inequalities.” Source page.
4. Digital Public Goods Alliance. “Digital Public Goods Standard.” Source page.
5. Open Source Initiative. “The Open Source Definition.” Source page.